lenguajes

DE PROGRAMACIÓN

Es un conjunto de reglas sintácticas, semánticas y símbolos formales que permite a un ser humano escribir instrucciones estructuradas para que una computadora las interprete, procese y ejecute.

Elementos esenciales

DE UN LENGUAJE DE PROGRAMACION:

SINTAXIS: Es el conjunto de reglas gramaticales que dictan como deben escribirse las palabras, simbolos y estructuras para que el código sea válido.

SEMANTICA: Es el significado o la acción lógica que realiza cada instrucción escrita.

VOCABULARIO / PALABRAS RESERVADAS: Palabras clave con significado especial para el lenguaje que no pueden usarse para otros fines. Ejemplo: while, return, etc.

PROGRAMACIÓN: Es el proceso de diseñar, escribir, probar y mantener código fuente para dar instrucciones a una computadora permitiendo ejecutar tareas especificas o resolver problemas de forma automatizada.

ALGORITMO: Es una secuencia finita, ordenada y lógica de pasos o instrucciones precisas que deben seguirse para realizar una tarea o resolver un problema concreto.

SMART | HIGHCLASS

Lenguaje de Alto Nivel

Es un lenguaje diseñado para ser leído y escrito fácilmente por los seres humanos. Utiliza palabras en ingles y estructuras lógicas cercanas al lenguajes nativo, independizando al programador de la arquitectura interna de hardware. Ejemplo: C , C++ , Python , JavaScript.

Lenguaje de Bajo Nivel

Es un lenguaje que opera con mínima o nula abstracción del hardware, diseñado para comunicarse e interactuar directamente con la memoria y el procesador de la computadora. Ejemplo: lenguaje máquina y lenguaje ensamblador.

Entorno de Desarrollo

INTEGRADO (IDE)

Es una aplicación de software que reune todas las herramientas necesarias para que un desarrollador escriba y pruebe código en un solo lugar.

COMPONENTES PRINCIPALES DE UN IDE

EDITOR DE CÓDIGO FUENTE: Incluye resaltado de sintaxis autocompletado y detección de errores en tiempo real.

COMPILADOR / INTERPRETADOR INTEGRADO: Permite ejecutar el programa con un solo click.

DEPURADOR (DEBUGGER): Herramienta para inspeccionar la memoria, avanzar línea por línea y rastrear errores.

GESTOR DE PROYECTOS: Organiza múltiples archivos, bibliotecas y configuraciones del programa.

SMART HIGHCLASS

VARIABLE: Una variable es un espacio reservado en la memoria RAM del ordenador que se utiliza para almacenar un valor que puede cambiar a lo largo de la ejecución del programa. Tiene un nombre (Identificador) y un tipo de dato asociado. Ejemplo:

- Num1 = 5
  base = 10

- Convertir grados Celsius a grados Fahrenheit
  °F = (°C × 1.8) + 32

CONSTANTE: Una constante es un valor almacenado en memoria que no puede cambiar ni ser modificado una vez que a sido definido durante la ejecución del programa. Se utilizan para evitar errores y representar valores fijos del dominio.

TIPOS DE DATOS:

CATEGORIA	TIPO DE DATO	TAMAÑO HABITUAL	RANGO
ENTERO	int	4 bytes	-2,147 483 648 a 2 147 483 647
	Short	2 bytes	-32 768 a 32 767
	long	4 a 8 bytes	-2 147 483 648 a 2147 483 647 / ±9 223 372 036 854 775 807
	long long	8 bytes	-9 223 372 036 854 775 808 a 9 223 372 036 854 775 807
PUNTO FLOTANTE	float	4 bytes	$\pm 3.4 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$
	double	8 bytes	$\pm 1.7 \times 10^{-308}$ a $\pm 1.7 \times 10^{308}$
CARACTERES	char	1 byte	-128 a 125
	wchar_t	2 a 4 bytes	0 a 65 535 0 a 4 294 967 295
LÓGICOS	bool	1 byte	0 a 1 (true o false)

SMART HIGHCLASS

DFD

Es una representación gráfica que muestra como fluye la información a través de un sistema o proceso.

ELEMENTOS PRINCIPALES:

Símbolo	Nombre	FUNCIÓN
Cuadro / Rectangulo	Entidad Externa	Representa la fuente u origen de los datos
Circulo / Ovalo	Proceso	Operación o función que transforma los datos
Lineas Horizontales / Rectangulo abierto	Almacén de datos	Lugar donde se guardan los datos temporal o permanentemente
Flecha	Flujo de Datos	Muestra la direcciones en lo que viaja la información entre los elementos

SMART | HIGHCLASS

Pseint

Es una herramienta educativa de Software libre diseñada para ayudar a estudiantes a dar sus primeros pasos en la programación y el desarrollo de la lógica algorítmica.

Utiliza pseudocodigo en español, lo que permite centrarse en los conceptos fundamentales de la programación (variables, condicionales, bucles y funciones) sin la compilación de la sintaxis estricta de un lenguaje real (como C++, Java o Python)

CONVERSIÓN DE TIEMPO (Segundo a minutos / horas):
Leer un valor en segundos e imprime a cuantos minutos y horas equivale.

CALCULO IGV DE UN PRODUCTO:
Solicita el precio base de un producto y calcula el monto del IGV (18%) junto con el precio total final.

AREA Y PERIMETRO DE UN RECTANGULO:
Pide la longitud de la base y la altura de un rectangulo para imprimir su area y perimetro.

SMART | HIGHCLASS

Operadores

OPERADORES ARITMETICOS

+ , - , * , / , % , ++, --

OPERADORES RELACIONALES

== , != , > , < , >= , <=

OPERADORES LÓGICOS

&& , || , ! , AND , O , NOT

If - else: Las estructuras if-else son sentencias de control condicionales que permiten tomar decisiones en el código ejecutando bloques de instrucción unicamente cuando se cumple una condición

Estructura If (Condicional Simple) Evalua una condición. Si la condición resulta verdadera (true), se ejecuta el codigo dentro del blogue. Si es falsa (false), el programa emite ese blogue y continua con la siguiente instrucción.

Estructura If - else (Condicional Doble) Permite definir en camino alternativo Si la condición resulta verdadera (true) se ejecuta el primer blogue. Si es falso (false), se ejecuta obligatoriamente el blogue dentro del else.

SMART | HIGHCLASS

Switch - case:

En C++, switch / case, es una estructura de control de selección multiple que evalua el valor de una variable y ejecuta un bloque de código según la coincidencia con una lista de casos enteros o caracteres.

Ejemplo:

```
switch: (variable) {
        case: (var 1)
              // ejecuta un código:
                 break;
        case: (var 2)
              // ejecuta un codigo;
                 break;
        default:
                 codigo no coincide con ningun previo case;
                 break;
}
```

- Crear un codigo C++ para la clasificación de glucosa en la sangre para la detección de hipoglucemia, diabetes o pre-diabetes.

SMART | HIGHCLASS

ESTRUCTURAS repetitivas

También llamada bucle ciclo o iteración es un bucle de código que permite ejecutar una o varias instrucciones de forma recurrente mientras se cumpla una determinada condición lógica.

Su proposito principal es automatizar tareas repetitivas, evitar la duplicidad de código y procesar colecciones de datos (como lista o arreglos) de manera eficiente.

```
for (inicio ; final ; incremento)

for (i=1 ; i<=10 ; i++)
```

Iteración: Una iteración es cada una de las repeticiones o ejecuciones o individuales de un bucle. Si un bucle se ejecuta 5 veces, se dice que se realiza 5 iteraciones.

Contador: Es una variable entera cuyo valor se incrementa o decrementa en una cantidad constante (generalmente de 1 en 1) en cada iteración. Se utiliza para registrar la cantidad de veces que ocurre un evento o se repite un ciclo.

```
contador = contador + 1
contadorpar = contadorpar + 1
contarPar = contarPar + 1
contarpar = contarpar + 1
```

Acumulador: Es una variable que almacena la suma o multiplicación sucesiva de valores variables en cada iteración. Se utiliza para obtener totales, promedios o sumatorias acumuladas.

SMART | HIGHCLASS

Suma = Suma + i;

Suma += i;

Control de flujo

ITERATIVO

Ctt proporciona sentencias especiales para llevar la ejecución normal de una estructura repetitiva

Break: Interrumpe y sale inmediatamente del bucle, ignorando las iteraciones restantes.

Continue: Salta el resto de las instrucciones de la iteración actual y pasa directamente a la siguiente iteración (evaluando de nuevo la condición de incremento).

TABLA DE MULTIPLICACIÓN:

1 x 1 = 1	2 x 1 = 2	10 x 1 = 10
1 x 2 = 2	2 x 2 = 4	10 x 2 = 20
⋮	⋮	⋮
1 x 10 = 10	2 x 10 = 20	10 x 10 = 100

```
For (i = 1 ; i<=10 ; i++)

    For (j = 1 ; j<= 10 ; j++)
    cout << i << "x" << j << " = " << i * j << endl;
```

SMART | HIGHCLASS

Estructura:

Estructura Repetitiva While (mientras)

Es un bucle o ciclo de control que ejecuta un bloque de código de forma iterativa mientras se cumpla una condición lógica determinada.

A diferencia de otros bucles como (for) el bucle while se utiliza principalmente cuando no se conoce con exactitud el numero exacto de iteración que se van a realizar antes de iniciar la ejecución:

Funcionamiento Básico:

1. Evaluación: Antes de ejecutar el código se evalua la condición lógica.
2. Ejecución: Si la condición es verdadera (true), se ejecuta el cuerpo del bucle.
3. Repetición: Al terminar el bloque, se vuelve a evaluar la condición el proceso continua hasta que la condición pase a ser falsa (False).
4. Finalización: Si la condición es falsa desde el inicio, el bloque dentro del bucle no se ejecutara ni una sola vez.

Estructura Repetitiva Do-While

Las estructuras repetitivas do-while (hacer- mientras) es un bucle o ciclo de control que ejecuta un bloque de codigo al menos una vez y luego continua repitiendolo mientras se cumpla una condición logica determinada.

SMART | HIGHCLASS

Funcionamiento básico:

- Ejecución obligatoria: El programa entra al ciclo y ejecuta las instrucciones del bloque directamente, sin comprobar ninguna condición previa.
- Evaluación posterior: Al llegar al final del bloque, se evalua la condición lógica expresada en el while.
- Repetición: Si la condición es verdadera (true), el flujo regresa al inicio del bloque para repetir el proceso.
- Finalización: Si la condición es falsa (False), el ciclo termina y el programa continua con la instrucción siguiente.

SMART | HIGHCLASS

ARREGLOS

Un arreglo (también conocido como array, vector o matriz) es una estructura de datos homogenea y de tamaño fijo que permite almacenar una colección de elementos del mismo tipo bajo un unico nombre, cada elemento individual dentro del arreglo se identifica y se accede medrante 1 o más indices numéricos

Arreglo Unidimensional
(VECTOR)

Un arreglo unidimimensional es una colección lineal de datos que se organizan de forma secuencial, uno al lado del otro en la memoria del computador. se puede visualizar como una sola fila con múltiples casillas.

Ejemplo:

0	1	2	3	4
2	14	15	18	20

TIPO DE DATO: int

int notas [5];

- Leer n notas mostrar las notas ingresadas y obtener el promedio.

Arreglo Bidimensional
(MATRIZ)

Un arreglo bidimensional es una estructura organizada en una tabla de dos dimensiones, filas y columnas. Se puede imaginar como una cuadricula o una hoja de calculo, donde la intersección de una fila y una columna determina una posición o casilla especifica.

Ejm:

14	15	16
8	13	20

int matriz [2] [3];

- Leer una matriz de orden n por n se muestra la matriz y luego sumar sus elementos

SMART HIGHCLASS